



◆ ◆ 所属专栏：[Linux](#) ◆ ◆

◆ ◆ 作者主页：[某某](#) ◆ ◆

Linux：进程间通信

介绍：

目的：

- 数据传输，一个进程需要将它的数据发送给另一个进程
- 资源共享，多个进程间共享同样的资源
- 通知事件，一个进程需要向另一个或一组进程发送消息，通知它（们）发生了某种事件（如进程终止通知父进程）
- 进程控制，有些进程希望完全控制另一个进程的执行（如Debug进程），此时控制进程希望能够拦截另一个进程的所有陷入和异常，并能够及时知道它的状态改变

通信方式及发展：

- 管道

匿名管道

命名管道

- System V IPC

System V 消息队列

System V 共享内存

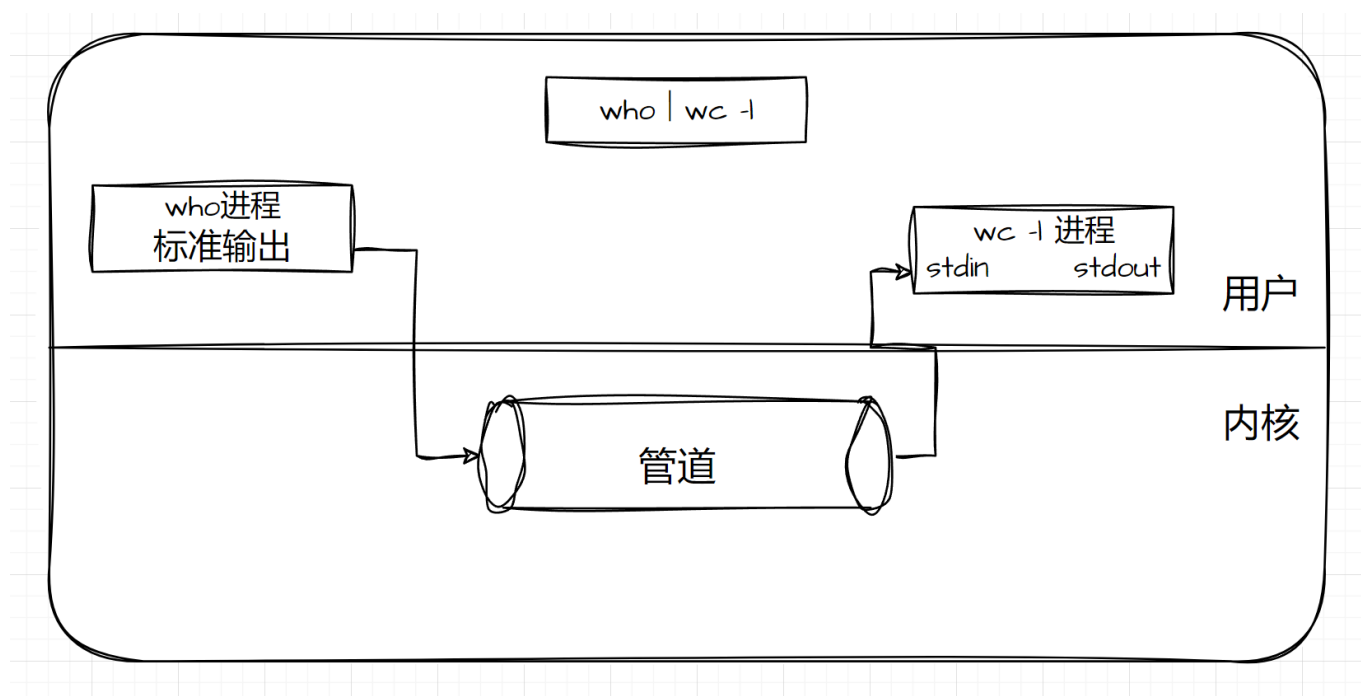
System V信号量

- POSIX IPC

- 消息队列
- 共享内存
- 信号量
- 互斥量
- 条件变量
- 读写锁

管道

管道是Unix比较古老的一种进程间通信的形式，我们把一个进程连接到另一个数据流成为一个“管道”



匿名管道

```
#include <unistd.h>
//功能：创建一匿名管道
//函数原型：
int pipe(int fd[2]);

//参数：
//fd：文件描述符数组，其中fd[0]表示读端，fd[1]表示写端
//返回值：成功返回0，失败返回错误代码
```

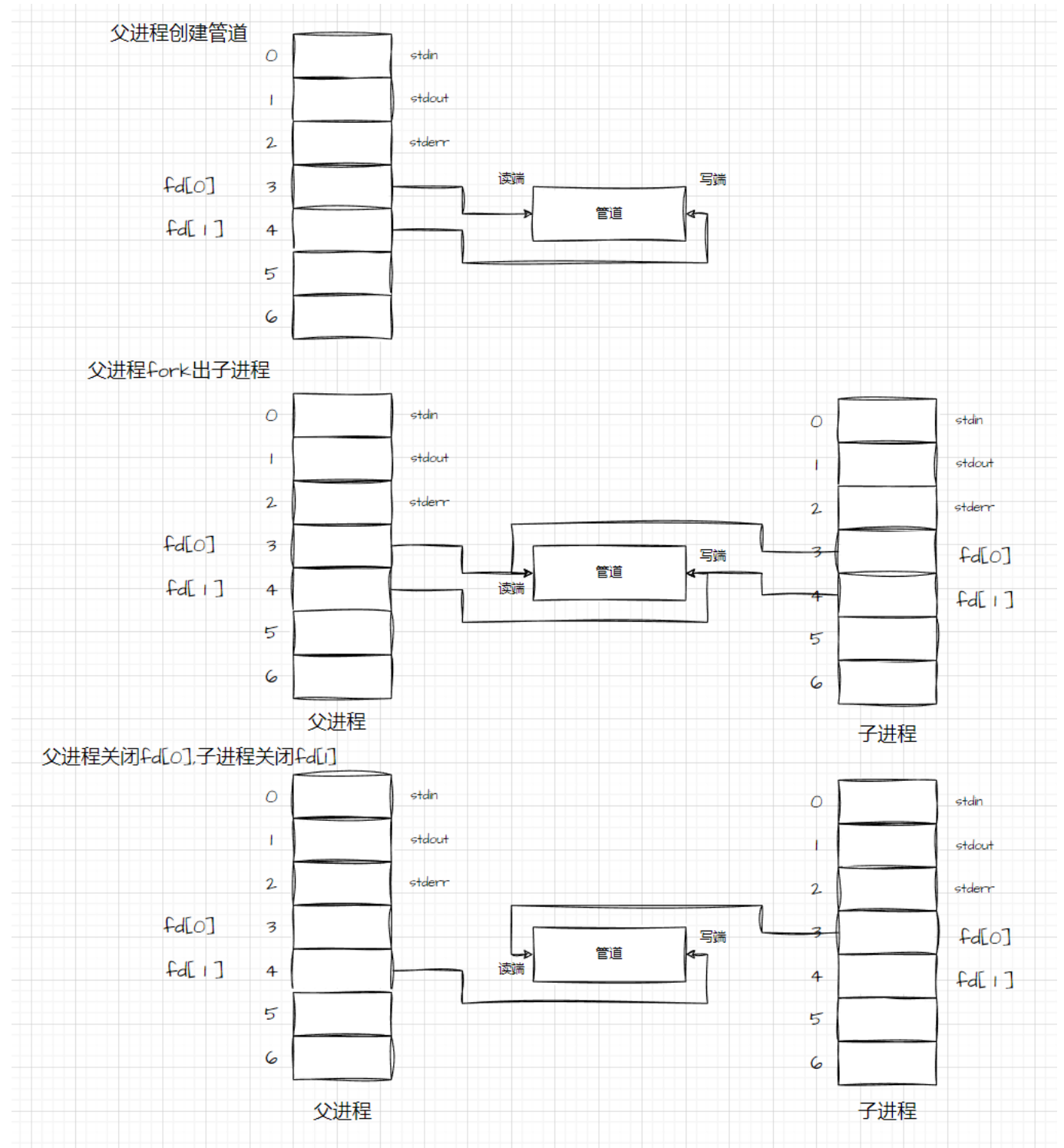
例子：可以使用管道从键盘stdin读取数据，写到屏幕stdout，具体代码如下

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
int main(void)
{
    int fds[2];
    char buf[100];
    int len;
    if (pipe(fds) == -1)
        perror("make pipe"), exit(1);
    // read from stdin
    while (fgets(buf, 100, stdin))
    {
        len = strlen(buf);
        // write into pipe
        if (write(fds[1], buf, len) != len)
        {
            perror("write to pipe");
            break;
            memset(buf, 0x00, sizeof(buf));

            // read from pipe
            if ((len = read(fds[0], buf, 100)) == -1)
            {
                perror("read from pipe");
                break;
            }

            // write to stdout
            if (write(1, buf, len) != len)
            {
                perror("write to stdout");
                break;
            }
        }
    }
}
```

另外pipe可以用于父子之间通信，fork后，子进程会继承父进程的文件描述符，父子进程分别关闭对应的读或写端就可以实现单向通信



我们在操作管道的时候，操作的是文件描述符。那么，匿名管道是文件吗？

首先匿名管道在磁盘中并没有对应的空间，只是在内核层面维护着一段缓冲区（内存上），但是它又继承了一些文件的操作，比如可以使用系统调用`read`、`write`对其进行读写操作。总之匿名管道不是实体文件，但是在行为上和真正的文件相似，这也迎合了Linux下一切皆文件的思想。

管道的读写规则

- 当没有数据可读时：

`O_NONBLOCK` disable（禁用非阻塞模式）：`read`调用时阻塞，即进程暂停执行，一直等到有数据来为止。

O_NONBLOCK enable (使用非阻塞模式) : `read`调用返回-1, `error`值为EAGAIN。

- 当管道满的时候:

O_NONBLOCK disable (禁用非阻塞模式) : `write`调用时阻塞, 直到有进程来读走数据。

O_NONBLOCK enable (使用非阻塞模式) : `write`调用返回-1, `error`值为EAGAIN。

- 如果所有管道写端对应的文件描述符被关闭, `read`返回0。
- 如果所有管道读端对应的文件描述符被关闭, 则`write`操作会产生信号SIGPIPE, 进而可能导致`write`进程退出。
- 当要写入的数据量不大于PIPE_BUF时, Linux将保证写入的原子性。
- 当要写入的数据量大于PIPE_BUF时, Linux将不再保证写入的原子性。

管道特点

- 只能用于具有**共同祖先**的进程 (或者是具有**血缘关系**的进程) 之间进行通信; 通常, 一个管道由一个进程创建, 然后该进程调用fork, 此后父子进程之间就可应用该管道。
- 管道提供流式服务
- 管道的生命周期是随着进程的, 进程退出, 管道就释放了。
- 内核会对管道进行同步和互斥
- 管道通信时**半双工**的, 数据只能向一个方向流通; 需要双方通信时, 需建立两个管道。

基于匿名管道的进程池

这个小项目, 就是先创建一个父进程, 之后fork出多个子进程, 并分别创建管道。当父进程收到任务时, 可以通过朝对应管道发送任务码, 从而控制对应的子进程去完成任务, 实现了任务分配。

任务分配时该选哪一个子进程呢? 轮询, 随机, 还是给每一个进程tag一个任务量, 不同的方法有不同的好处。

另外要注意, 之前父进程开的管道的读端会继承给下一个子进程, 这样第一个子进程对应管道的读端就会有多个读端, 引用计数随着进程的增多不断增多。这是一个藏的比较深的bug当时如果不是蛋哥说出来我一定不知道。解决方法是在fork新的子进程时删除上一个子进程对应管道的写端。这样每一个子进程对应的管道的读端引用计数都是1。在删除的时候本该是关掉对应的写端, 读端读到0然后退出之后父进程waitpid回收资源拿到退出码。如果读端的引用计数不是1, 就会出现关闭父进程的写端后, 子进程不会退出, 一直在read那里阻塞。

具体信息可以参考源代码: [25/Process_Pool · 钦某/Code - 码云 - 开源中国 \(gitee.com\)](#)

命名管道

匿名管道的限制就是只能在具有共同祖先 (亲缘关系) 的进程间通信。如果我们想在不相关的进程间交换数据, 就可以使用FIFO文件来进行, 它被叫做命名管道。本质上也是文件, 有文件名, 所以叫做命名管道

命名管道可以在命令行上创建:

```
$ mkfifo filename
```

命名管道可以代码里创建，相关函数：

```
int mkfifo(const char* filename, mode_t mode); // 这个函数在手册3中，不属于系统调用

int main()
{
    mkfifo("p2", 0644);
    return 0;
}
```

匿名管道与命名管道的区别

特性	匿名管道	命名管道
通信范围	亲缘关系进程	任意进程（可跨网络）
通信方向	半双工	全双工
存在形式	内存缓冲区，无文件实体	文件系统路径（如FIFO文件）
创建方式	pipe()	mkfifo()或系统API
生命周期	随进程结束销毁	需显式删除文件
典型应用场景	父子进程快速通信	多进程协作、服务端-客户端

命名管道的打开规则

如果当前打开操作是为读而打开FIFO时

O_NONBLOCK disable（禁用非阻塞模式）：阻塞直到有相应进程为写而打开该FIFO

O_NONBLOCK enable（使用非阻塞模式）：立刻返回成功

如果当前打开操作是为写而打开FIFO时

O_NONBLOCK disable（禁用非阻塞模式）：阻塞直到有相应进程为读而打开该FIFO

O_NONBLOCK enable（使用非阻塞模式）：立刻返回失败，错误码为ENXIO

基于命名管道的进程间通信封装

这个项目里只有一个类，这个类里面只有三个开放的成员函数：

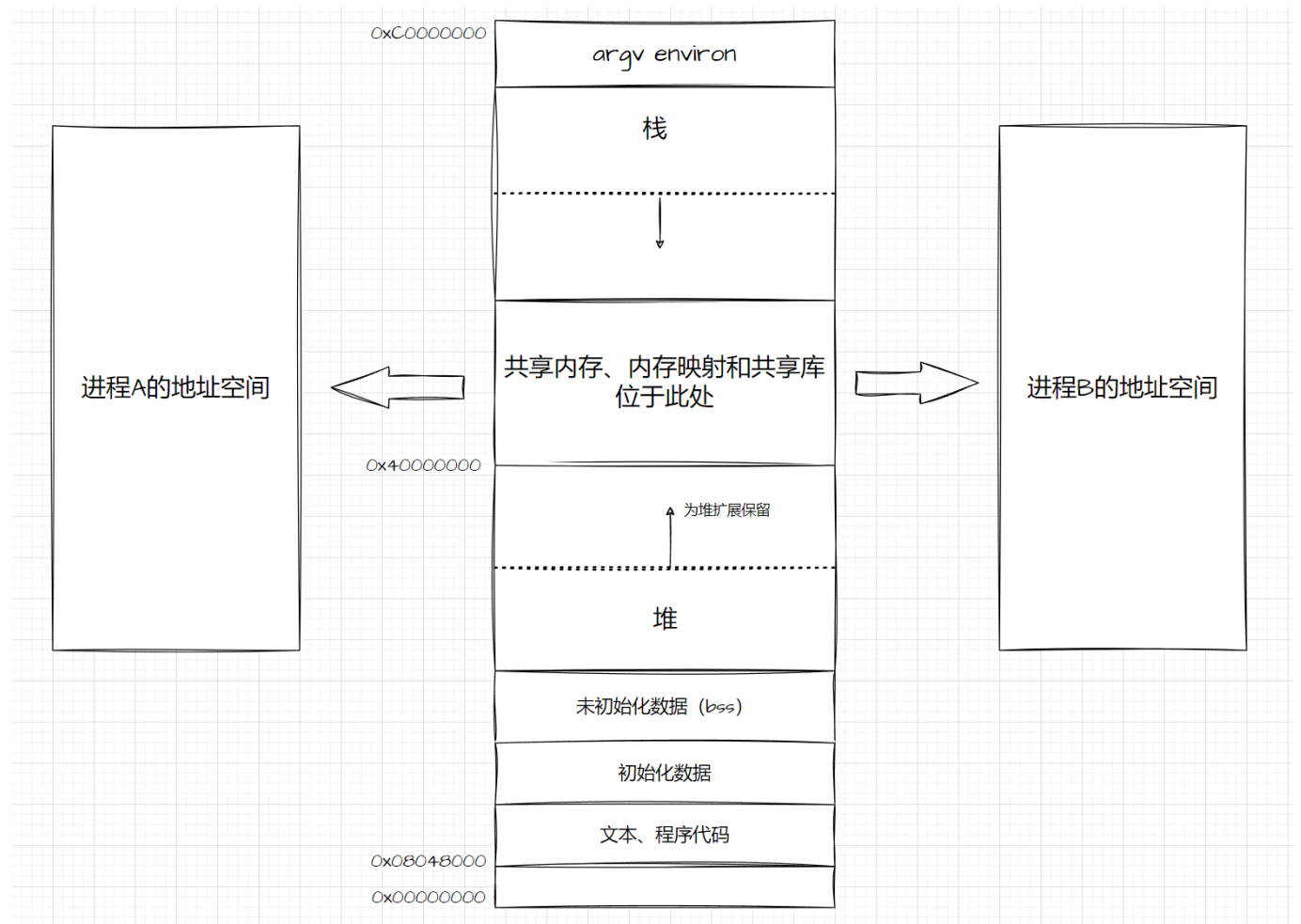
- 1. 打开管道文件
- 2. 对管道文件进行操作
- 3. 获取管道文件的文件描述符fd

进程在初始化类时会传入一个参数，这个参数由宏定义，`SERVER`和`CLIENT`，初始化后，系统会自动调用构造函数，并把`SERVER`或`CLIENT`赋值给成员变量，方便后续进行条件编译。在构造函数里面条件调用了一个创建命名管道文件的私有函数。在server端初始化时会调用这个函数。而在client端则不会调用该函数。

在Operate函数中也使用了条件编译，如果成员变量为`SERVER`则调用`Read`函数从管道里面读取数据，如果成员变量为`CLIENT`则调用`Write`对管道进行写入。在该类中，并没有对`Read`和`Write`函数做高耦合。这也许会是后续复用代码时需要改进的方向之一。

system V 共享内存

共享内存区是最快的IPC形式。一旦这样的内存映射到共享它的进程的地址空间，这些进程间数据传递不再涉及到内核，换句话说，进程不在通过调用进入内核的系统调用来传递数据



共享内存数据结构struct

在操作系统中不仅仅只有两个进程通过共享内存相互通信，那么操作系统就必须要将这么多的共享内存管理起来。那么又是这个老生常谈的问题，**先描述再组织。**所以内核中组织共享内存的结构体就应运而生了。

结构体里面记录了权限、大小、最后一次关联时间、最后一次改变时间、创建进程的pid、最后一个操作进程的pid等信息。

```
/* Obsolete, used only for backwards compatibility and libc5 compiles */
struct shmid_ds {
    struct ipc_perm    shm_perm; /* operation perms */
    int                shm_segsz; /* size of segment (bytes) */
```

```
__kernel_time_t    shm_atime; /* last attach time */
__kernel_time_t    shm_dtime; /* last detach time */
__kernel_time_t    shm_ctime; /* last change time */
__kernel_ipc_pid_t shm_cpuid; /* pid of creator */
__kernel_ipc_pid_t shm_lpid; /* pid of last operator */
unsigned short      shm_nattch; /* no. of current attaches */
unsigned short      shm_unused; /* compatibility */
void                *shm_unused2; /* ditto - used by DIPC */
void                *shm_unused3; /* unused */
};
```

共享内存函数

- `shmget`函数

功能：用来创建共享内存

原型：

```
int shmget(key_t key, size_t size, int shmflg);
```

参数：

`key`：这个共享内存段名字，由`ftok`函数规定获取

`size`：共享内存大小

`shmflg`：由九个权限标志构成，它们的用法和创建文件时使用的`mode`模式标志是一样的，取值为
IPC_CREAT：共享内存不存在，创建并返回；共享内存已存在，获取并返回。取值为IPC_CREAT |
IPC_EXCL共享内存不存在，创建并返回；共享内存已存在，出错返回。

返回值：成功则返回一个非负整数，即该共享内存段的标识码；失败返回-1。

- `shmat`函数

功能：将共享内存段连接到进程地址空间

原型：

```
void *shmat(int shm_id, const void *shmaddr, int shmflg);
```

参数：

`shm_id`：共享内存标识

`shmaddr`：指定连接的地址

`shmflg`：它的两个可能取值是SHM_RND和SHM_RDONLY

说明:

`shmaddr`为NULL, 核心自动选择一个地址

`shmaddr`不为NULL切`shmflg`无SHM_RND标记, 则以`shmaddr`为连接地址

`shmaddr`不为NULL且`shmflg`设置了SHM_RND标记, 则连接的地址回自动向下调整为SHMLBA的整数倍

公式: $shmaddr - (shmaddr \% SHMLBA)$

`shmflg = SHM_RDONLY`, 表示连接操作用来只读共享内存

返回值: 成功返回一个指针, 指向共享内存的第一个节; 失败返回-1

• `shmdt`函数

功能: 将共享内存段与当前进程脱离

原型:

```
int shmdt(const void *shmaddr);
```

参数: `shmaddr`是由`shmat`返回的指针

返回值: 成功返回0; 失败返回-1

注意: 将共享内存段与当前进程脱离不等于删除共享内存段

• `shmctl`函数

功能: 用于控制共享内存

原型:

```
int shmctl(int shmid, int cmd, struct shm_ds *buf);
```

参数:

`shmid`: 由`shmget`返回的共相内存标识码

`cmd`: 将要采取的动作 (三个可取值)

命令	说明
IPC_STAT	把shmid_ds结构中的数据设置为共享内存的当前关联值
IPC_SET	在进程由足够权限的前提下, 把共享内存的当前关联值设置为shmid_ds数据结构中给出的值
IPC_RMID	删除共享内存段

`buf`: 指向一个保存着共享内存的模式状态和访问权限的数据结构

返回值: 成功返回0; 失败返回-1

基于system V共享内存的进程间通信

在这个项目中只有一个类`Shm`且只有一个方法就是获取共享内存的地址, 也就是调用了函数`shmat`并返回。其他的方法都被封装到了构造函数和析构函数中。用户只需要在初始化阶段传入一个参数用于指明`server`或`client`即可。

另外, 在此项目中, 还使用了命名管道(被修改过的命名管道类)进行进程间通信的第二信道, `client`进程结束前, 会通过第二信道发送一条指令, `server`在收到指令后会跳出循环, 调用类的析构函数, 结束进程。

详情可参考代码: [25/Share_memery · 钦某/Code - 码云 - 开源中国 \(gitee.com\)](#)

需要注意的是: 共享内存不像管道那样, 由同步和互斥机制, 这也会导致缺乏控制, 会带来并发问题, 但是有缺点就有优点, 它快啊!